Translate (Feature B) — v1 Build, Testing & Gotchas

Paragraph-only EN<->AR translation — backend build and manual testing, 2026-07-24

Contents

1. Scope decided for v1

Explicitly narrowed before any code was written: v1 is **paragraph-only** translation — paste English or Arabic text in, get the other language back. No document upload, no PDF/DOCX/text file parsing, no formatting-preserving reassembly, no freeform translation instructions ("keep font size X", "header color Y"). Those are real, named future phases, not cut features — just not this release. Frontend work (including the described grayed-out placeholder buttons for the deferred features) is deliberately deferred until all backend/API work across every feature is done — nothing in this document touches frontend code.

2. What was built

File	Purpose
`agents/translation/config.py`	`MODAL_TRANSLATE_URL` (env-driven, same pattern as the embedder/Qwen-lite config), `MAX_TEXT_LENGTH = 2000` characters.
`agents/translation/translate.py`	`translate_paragraph(text)` — detects English/Arabic via the same `detect_language()` the chatbot already uses, translates to the other language via the deployed Modal `Translator` (MADLAD-400) endpoint, returns `{translated_text, source_language, target_language}`.
`app/main.py`	`POST /api/translate` replaced from a mocked async job/poll pair with a real, synchronous, JWT-protected endpoint.
`docker-compose.yml`	`MIZAN_TRANSLATE_URL` added to the `chatbot` service's environment block.
`.env.example`	`MIZAN_TRANSLATE_URL` documented alongside the other two Modal endpoint vars.

The Modal-side model (`Translator` class, MADLAD-400, in `modal-serving/modal_app.py`) already existed from earlier work — this session only had to confirm it was actually deployed and wire the gateway up to call it.

3. Design decisions

Synchronous, not async job/poll

The mock this replaced modeled translation as a background job (`POST` returns a `job_id` + `status: queued`, `GET /api/translate/{job_id}` polls for the result) — a reasonable shape for translating a whole document, which could take a while. For paragraph-only v1, a single request/response (same shape as `POST /api/know-vat-zatca/chat`) is simpler and a better match for something that should feel instant. The async shape can come back when real document translation is built.

Auto-detected language, no picker

The endpoint takes only `{"text": "..."}` — no `target_language` field. Source language is detected from the pasted text itself and the target is always "the other one," matching the described UX exactly: paste English, see Arabic; paste Arabic, see English. Reuses the chatbot's existing `detect_language()` rather than adding a new detector.

Hard length cap instead of silent truncation

The Modal `Translator` endpoint takes one text blob with no internal chunking (`max_new_tokens=1024` on generation, confirmed by reading `modal-serving/common/model_loader.py`'s `run_translation()`). Since there's no document-splitting step in this v1, a request longer than a real paragraph would either get silently truncated by the model or produce degraded output. `MAX_TEXT_LENGTH = 2000` characters rejects that case early and explicitly (`422`) instead.

4. Manual testing results

Test	Input	Result
EN → AR	"Businesses must register for VAT once their taxable supplies exceed the threshold."	"ويجب على الشركات أن تسجل نفسها لضريبة القيمة المضافة بمجرد أن تتجاوز إمداداتها الخاضعة للضريبة العتبة." — correct, `source_language: "en"`, `target_language: "ar"`
AR → EN	"الفاتورة الإلكترونية إلزامية لجميع الشركات الخاضعة للضريبة."	"Electronic invoice is mandatory for all taxable companies." — correct, `source_language: "ar"`, `target_language: "en"`
Empty text	`{"text": ""}`	`422` — `"String should have at least 1 character"` (caught by Pydantic's `min_length=1` before reaching the translation logic at all)
Over length cap	764-word / 5,447-character passage	Expected `422` — `"Text exceeds the 2000-character paragraph limit"`

Confirmed end-to-end through the real HTTP API (auth → endpoint → Modal → response), not just Python-level unit checks — same standard applied to the RAG feature's testing.

5. Gotchas hit while testing (none are pipeline bugs)

Unescaped quotes inside a JSON string — the request itself was invalid, not a bug

WHAT HAPPENED

A test request was sent with a `text` value that itself contained literal double-quote characters:

```
{"text": " ... the "under construction" message are pure frontend ... "}
```

This is not valid JSON. A `"` character inside a JSON string must be escaped as `\"` — otherwise the parser reads the string as ending right before the word `under`, and everything after that point is no longer inside any recognized string or object field. The server correctly rejected it before any of this app's own code even ran:

```
{
  "detail": [
    {
      "type": "json_invalid",
      "loc": ["body", 96],
      "msg": "JSON decode error",
      "ctx": {"error": "Expecting ',' delimiter"}
    }
  ]
}
```

HOW TO AVOID IT

Two options when hand-writing a JSON request body (in Swagger, curl, or anywhere else) that contains quote characters in the text itself:

```
Escape them:
{"text": "the \"under construction\" message ..."}

Or just avoid double quotes in test content entirely
(swap to single quotes or plain wording) — simplest when the
quotes aren't semantically important to what you're testing:
{"text": "the under-construction message ..."}
```

This applies to any endpoint that accepts a JSON body with free-text fields, not just `/api/translate` — worth knowing generally, not a translation-specific issue.

Windows console can't print Arabic output — not a request failure

WHAT HAPPENED

Running a quick Python check from a Windows terminal against a successful translation raised:

```
UnicodeEncodeError: 'charmap' codec can't encode characters in position 25-30: character maps to <undefined>
```

The translation call itself had already succeeded — this is the terminal's default `cp1252` codepage failing to *display* Arabic characters, not an error from the API or the translation pipeline. Same class of issue hit during the RAG feature's Arabic testing last session.

FIX

```
set PYTHONIOENCODING=utf-8   (or, in Git Bash: PYTHONIOENCODING=utf-8 python ...)
```

Swagger's own UI in a browser doesn't have this problem — it only bites ad hoc scripted checks run from a Windows terminal.

Occasional hallucinated leading token from the model

OBSERVED, NOT BLOCKING

One of four test translations came back with a spurious `"10- "` prefix that wasn't in the source text at all:

```
Input:  "The mandatory VAT registration threshold is important for businesses."
Output: "10- وتعتبر العتبة الإلزامية لتسجيل ضريبة القيمة المضافة هامة بالنسبة للأعمال التجارية."
```

The other three tests (including a retry of a similar sentence) came back clean. Same general category of issue as the occasional garbled leading tokens seen once from the RAG feature's Qwen-lite generation — an inherent model-quality quirk, not something wrong in this gateway's code. Not investigated further this session; worth watching if it recurs at a higher rate under real use.

6. Frontend placeholders — decided NOT a backend concern

The original ask included showing dummy/grayed-out UI elements for the deferred features (document upload, supported-file-types indicator, a freeform instructions bar) with an "under construction" message on click. Discussed and explicitly decided: **nothing is required from the backend for this.** Those are purely presentational — no data to fetch, no state to track, nothing that varies per user or request, since the features genuinely don't exist yet. A future frontend can hardcode "these are disabled" directly.

One optional idea floated and explicitly **not** pursued now: a small `GET /api/translate/capabilities` endpoint reporting which sub-features are `"available"` vs `"coming_soon"`, so a future frontend reads

feature status instead of hardcoding it, and flipping a feature on later needs no frontend deploy. Deferred as speculative work against a frontend that doesn't exist yet and whose real needs aren't known — revisit if/when frontend work actually starts.

7. Commands reference

Rebuild after a code change

```
docker compose up -d --build chatbot --no-deps
```

Confirm clean startup — look for these lines in the logs:

```
docker compose logs chatbot --since=1m
# INFO:app.main:Service is up.
# INFO:     Uvicorn running on http://0.0.0.0:8000
```

Test via Swagger

1. `http://localhost:8000/docs`
2. `POST /api/auth/login` (or `/register`) → copy `jwt_token`
3. **Authorize** (padlock) → paste raw token, no `Bearer` prefix
4. `POST /api/translate` → body `{"text": "..."}` → Execute

Test via curl

```
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"TestPass123!"}' \
  | python -c "import json,sys; print(json.load(sys.stdin)['jwt_token'])")

# English/ASCII text: inline -d is fine
curl -s -X POST http://localhost:8000/api/translate \
  -H "Content-Type: application/json" -H "Authorization: Bearer $TOKEN" \
  -d '{"text":"Please submit your tax return before the deadline."}'

# Arabic text: MUST go through a file (Windows/Git Bash mangles inline non-ASCII —
# see the offline pipeline's testing PDF for the same gotcha in RAG testing)
echo '{"text": "الفاتورة الإلكترونية إلزامية لجميع الشركات."}' > q.json
curl -s -X POST http://localhost:8000/api/translate \
  -H "Content-Type: application/json; charset=utf-8" -H "Authorization: Bearer $TOKEN" \
  --data-binary "@q.json"
```

Test the Modal endpoint directly (bypass the gateway, fastest loop)

```
python -c "
from dotenv import load_dotenv
load_dotenv()
from agents.translation.translate import translate_paragraph
print(translate_paragraph('Please submit your tax return before the deadline.'))
"
```

Generated 2026-07-24 — Mizan.ai, Translate (Feature B) v1 backend build and testing.